

You must read this document carefully

Fall 2026
CMPS310 Software Engineering
- Group Project -

Milestone-1: Software Requirements Analysis, Design, and Agile Project Management (10%)

This is a compulsory project. If you do not submit this, your grade will be zero (0) in this course.
Each group can have four members

Announcement Date: September 13, 2026

Submission Due Date: Midnight 23:59 October 18, 2026

PearlMart Inventory Management System

Objectives of the Project

The purpose of this project is to reinforce the concepts discussed in class. It provides an opportunity to apply the concepts and techniques learned to a realistic problem. The main objectives are to practice Requirements Specification and Object-oriented Analysis and Design Modeling for a software solution, and to gain practical experience in planning, organizing, tracking, and evaluating software project work using the Scrum agile process and a software project management tool.

Background

Assume that Ms. Sara Ahmed has recently been appointed as the operations manager of **PearlMart**, a medium-sized retail store. She has contacted your team to design a software system exclusively for managing the store's inventory activities.

PearlMart currently relies heavily on manual procedures to maintain information about goods. This has made it difficult to search for products, determine available quantities, update prices, identify expired goods, and retrieve previous transaction records. Manual processing has also resulted in occasional human errors, inconsistent records, and loss of information. Ms. Sara therefore requests the development of an automated **PearlMart Inventory Management System** that operates according to the requirements described below.

The system will support activities performed both in the warehouse and at the store's billing counters. Although the system can be accessed from different locations within PearlMart, it is considered **one integrated system**, and all inventory information should remain consistent.

Each item maintained by PearlMart has a unique item identifier and barcode. Information recorded about an item includes its name, selling price, available quantity, manufacturer, place of origin, and, when applicable, its expiration date. Additional comments about an item may also be recorded.

Several types of employees use the system. An **Administrator** performs administrative activities and manages important changes to system information. An **Inventory Manager** monitors the inventory and manages inventory-related transactions. A **Store Manager** supervises the receipt and storage of goods in the warehouse. **Salespersons** use the system mainly at the billing counters to process customer purchases and related transactions.

All authorized users must log in before using the system. A user enters a username and password, and the system verifies the credentials. If the credentials are valid, the user is given access to the functions appropriate to his/her responsibilities. Authorized users may also change their passwords.

When new goods arrive at PearlMart, an authorized employee scans the item's barcode or enters its identifier. If the item is new, the employee enters the required information about the item, including its name, selling price, quantity, manufacturer, place of origin, and expiration date when applicable. The system then creates a new inventory record. If the item already exists, its available quantity is increased accordingly.

Goods may also need to be removed from the inventory. An authorized employee identifies the item and specifies the quantity to be removed. The system retrieves the corresponding item information and checks whether the requested quantity is available. If the request can be processed, the system reduces the available quantity and records the change.

For goods that have expiration dates, the system keeps track of expiration information. Authorized employees can request information about expired goods. The system identifies the expired items and displays their details. The employee may then remove the appropriate goods from the inventory.

Authorized employees can search for an item using its identifier, barcode, or name. The system retrieves and displays the corresponding information, including the current price and available quantity. Certain item information may also be updated when necessary. For example, when the selling price of an item changes, an authorized employee retrieves the item, enters the new price, and the system updates the item record. PearlMart obtains goods from external **Suppliers**. When additional goods are required, the Inventory Manager prepares an order specifying the required item, quantity, delivery address, contact information, and any relevant comments. The system records the order and forwards the necessary order information to the selected Supplier.

The Supplier processes the request and provides information concerning the ordered goods and the corresponding invoice. The invoice contains information such as the ordered items, quantities, prices, and total amount. When the ordered goods are delivered to PearlMart, the Store Manager verifies the received goods. The system records the received quantities and updates the inventory accordingly.

At the billing counter, a Salesperson processes the items selected by a customer. For each item, the Salesperson scans the item's barcode or enters its identifier and specifies the quantity being purchased. The system retrieves the item information and checks whether the requested quantity is available.

For every valid item, the system determines its selling amount. Where applicable, discounts and taxes are calculated. After all selected items have been processed, the system calculates the final amount to be paid by the customer.

The system then generates a receipt containing relevant transaction information, including the purchased items, quantities, prices, discounts, taxes, and final amount. When the transaction is successfully completed, the quantities of the purchased items are deducted from the inventory.

Before a billing transaction is completed, the Salesperson may cancel the current receipt. If the receipt is cancelled, the current transaction is terminated and the quantities of the items involved must not be permanently deducted from the inventory. The system records the appropriate information concerning the cancelled transaction.

PearlMart also allows customers to request replacement of purchased items according to store policy. To process a replacement, the Salesperson enters the required transaction and item information. The system

retrieves the corresponding information and verifies the request. If the replacement is accepted, the system records the replacement and makes the necessary updates to the inventory.

A customer may sometimes request an item that is not currently available at PearlMart. In this case, an authorized employee records the requested item and the required quantity. The system maintains these customer requests so that the Inventory Manager can review them when deciding which goods should be ordered from Suppliers.

The system keeps track of important inventory activities and transactions, including goods added to or removed from inventory, goods received from Suppliers, supplier orders, sales transactions, cancelled receipts, replacement transactions, customer requests, and expired goods.

Authorized managers can also request **summary reports** about PearlMart's inventory activities. Depending on the type of report requested, the system retrieves the relevant information and generates a summary of the corresponding transactions and inventory records.

Ms. Sara expects PearlMart's inventory activities to be automated to the greatest extent possible. Information maintained by the system should remain consistent whether an operation is performed from the warehouse or from a billing counter.

The **Supplier is external to the PearlMart Inventory Management System**. Your analysis should therefore include the interaction between PearlMart and its Suppliers, but you should **not analyze or design the internal operations of a Supplier's own system**.

Additional Information

You will likely need additional information about the system's operation to complete this project. In software development, requirements may initially be incomplete or unclear, and software engineers often need to seek clarification or make reasonable assumptions.

For this project, your **Lab Instructor is your client**, and you may direct questions concerning the system requirements to him/her. You may also make logical assumptions whenever necessary to complete your analysis and design.

List and justify assumptions, identify affected models, and apply published course clarifications consistently.

Project Management Using Scrum

For Milestone 1, each group must:

Use the group repository already assigned to your team in cmpts310fall26 (<https://github.com/orgs/cmpts310fall26/repositories>). All four members must use their own accounts and follow Lab 2's GitHub workflow. Use GitHub Projects to plan and track the work, and Visual Paradigm to produce diagrams.

- Identify the Scrum responsibilities within the group, including the Scrum Master, Product Owner, and Developers.
- Agree on a Product Goal and Definition of Done: work is reviewed, corrected, consistent, and saved.
- Organize Milestone 1 into two Sprints, each no longer than two weeks, with dates and review checkpoints recorded at the start. For each Sprint, define a Sprint Goal and prepare a Sprint Backlog containing the selected work items.
- After each Sprint, provide a brief Sprint Review and Retrospective summary describing what was completed, any difficulties encountered, and improvements for the next Sprint.

- Track work through Todo, In Progress, In Review, and Done. Link each issue to a branch and pull request; obtain teammate review before merging to main. Each member must contribute and review substantive work. Coordinate edits to Word and .vpp files.
- Submit evidence of using Scrum with the Milestone-1 report, including screenshots or exported views of the Product Backlog, Sprint Backlog/Scrum board, task assignments, status updates, and progress. Include the GitHub Project URL.

Important: The Scrum records must reflect the actual progress of the project during the milestone. Evidence created only at the end of the milestone will not satisfy this requirement. The project management tool will be assessed as part of Milestone 1 and contributes 30% of the Milestone-1 grade (equivalent to 3 marks out of the 10 course marks allocated to Milestone 1).

Tasks of Your Project

Milestone 1: Software Requirements Analysis, Design, and Agile Project Management

1. Propose:
 - a. Develop logical data flow diagrams using the course convention: a Level 0 context diagram (one process, no internal stores), a Level 1 diagram of the major processes and logical data stores, and one Level 2 decomposition of a substantial Level 1 process. Use Gane–Sarson notation, meaningful directional data flows, and consistent process numbering. Demonstrate balancing between each parent and child diagram with a brief boundary-flow checklist (12%).
 - b. A use case diagram of the system (12%):
 - c. Compare what the DFD and use case diagram reveal about PearlMart. Explain their complementary strengths and limitations, and justify which is more useful for a specific analysis question. (4%)
2. Develop *use case specifications* for the four key and relatively complex use cases selected from your use case diagram. Each use case specification should include a brief description, primary actor(s), trigger, pre-condition(s), post-condition(s), normal scenario (actor action-system response table), and alternative flows (if any). Select four substantial business use cases involving validation, state changes, and meaningful alternative flows; routine login, search, or password changes do not satisfy this selection requirement. Use the use case specification template available from Blackboard (4 x 4%= 16%).
3. Using *object-oriented design fundamentals* (*objects, inheritance, abstraction, classification, encapsulation, modularity, responsibility-driven design, and justified relationships*),
 - a. Develop the *design class diagram* of the entire system with all required classes, their attributes and types, relationships (aggregation, generalization, association) with multiplicity where applicable, meaningful operations with parameter types and return types, and visibility of attributes and operations. Include domain classes and the application classes needed to allocate the specified responsibilities; implementation frameworks and database-specific classes are not required. Multiple readable views of the same consistent model are acceptable (22%).
 - b. Explain how your design assigns responsibilities and applies encapsulation, abstraction, and modularity, using examples from your classes. Use inheritance, aggregation, composition, or design patterns only where justified; no particular pattern is mandatory (4%).
4. Manage Milestone 1 using the Scrum agile process and a software project management tool (30%):

- a. Maintain prioritized GitHub Issues with acceptance criteria, one assignee, and type, priority, and phase labels. Link them to the Project and the Milestone-1 GitHub milestone, using the submission due date (7%).
- b. In the assigned repository, maintain a README with team members, GitHub usernames, Scrum responsibilities, and the Project link, plus an appropriate .gitignore (5%).
- c. Keep the board current and provide dated activity history or snapshots showing progress across each Sprint. Include issue-to-PR links and meaningful teammate reviews; a final board alone is insufficient (7%).
- d. Plan two Sprints of no more than two weeks each, both ending before the submission deadline, with goals, dates, and selected issues. Use a Sprint or Iteration field, estimates, and start/target dates. Check workload in Table view and the schedule in Roadmap view (7%).
- e. Summarize each Sprint's completed work, feedback, difficulties, and improvement action. Explain how Sprint 2 applied the first retrospective's improvement (4%).

Include your assumptions table and a compact traceability table linking major requirements to use cases, DFD processes, and responsible classes. Use consistent names and rules across artifacts; this evidence supports the existing modeling criteria and carries no additional marks. Assignment components will be evaluated for accuracy, clarity, relevancy, and completeness (especially among components and among artifacts) of your document.

- Each student must upload their electronic copy of Milestone-1 to Blackboard as a **Word Document and .vpp files** by the due date and time.
- Your word document and .vpp files must include all diagrams produced in Visual Paradigm.

By the deadline, each group must commit and push the final Word report, matching .vpp source files, and required project-management evidence to a Milestone-1 folder on the main branch of its instructor-provided GitHub repository. Every member must also upload the same final files individually on Blackboard. Include the repository URL in the report, and the final submission commit link and GitHub Project URL in each Blackboard submission. Both channels are required, and their final files must match. Include legible diagrams in the report and verify that the .vpp files open and match them.

Scrum project management evidence: Include the required Product Backlog, Sprint Goal(s), Sprint Backlog/Scrum board, Scrum responsibility assignments, task assignments, progress/status evidence, Sprint Review and Retrospective summary, and links to the instructor-provided group repository, GitHub Project, and Milestone-1. Include a Table view grouped by Assignees, a Roadmap view, and links tracing each member's substantive contribution from issue to reviewed pull request and final artifact. Assess contribution quality and review evidence, not raw commit counts. Keep the workspace private.

Grading scheme for Milestone-1: Requirements Analysis, Design, and Agile Project Management

Tasks	Criteria	Grade
1	(a) DFD (12%), (b) Use case diagram (12%), (c) Explanation (4%)	28
2	4 use case specifications using the template (4 x 4%=16%)	16
3	Design Class diagram of the entire system (22% for (a); 4% for (b))	26
4	Scrum planning, GitHub Issues and Projects, and traceable group collaboration	30
Total		100

More details will be provided for the subsequent milestones.

Submission requirements

Your assignment group/team MUST comply with the following submission requirements; **otherwise, the grade will be deducted:**

1. Your group should have four members. If you have less than **4** members in your group, please contact your lab instructor. You will form your own group.
2. The group must submit the final files to its assigned GitHub repository, and each member must upload the matching files individually on Blackboard by the same deadline. The cover page of your submission MUST contain the following items in four bullets:

[Each member of the group will be required to submit a peer group evaluation with an estimate of the contribution of each member to the project. Write the percentage contribution made by each team member so that it adds up to 100%. This evaluation *may* be used to adjust the marks awarded to each team member.]

- Effort distribution of the student:

SID: _____	STUDENT NAME: _____	Effort given _____	%
SID: _____	STUDENT NAME: _____	Effort given _____	%
SID: _____	STUDENT NAME: _____	Effort given _____	%
SID: _____	STUDENT NAME: _____	Effort given _____	%

- Course number _____
- Submission date _____
- Theory Class section: **L01 / L02 / L51 / L52** (select one)

- **DECLARATION:** *We hereby certify that no part of this project or product has been copied from any other student's work or from any other sources except where due acknowledgment is made in the project. No part of this project/product has been written/produced for us by any other persons.*

Note: If the above submission requirements are not met, up to 10% of marks will be deducted from this milestone.

3. Be aware of

- *Submitted work must be the student's own work*

You cannot copy the project from other groups.